ENGRAM V0.3.0 · SENTINEL

# When you give Claude Code a task, *seven hooks* decide what your agent **actually reads**.

The old ceiling was that your agent had to *remember* to use engram. The Sentinel removes that contract. It sits at the tool-call boundary and intercepts every Read, Edit, Write, and Bash — replacing full file reads with **~300-token structural graph summaries** before your agent ever sees the content.

| TOKEN REDUCTION | READ HIT RATE | HOOK HANDLERS | TESTS PASSING |
|---|---|---|---|
| 82% | 75% | 7 | 439 |

01 · THE LIFECYCLE

# Four events fire in order. Your agent never sees the seams.

From the moment you press Enter to the moment Claude writes back, engram is watching four Claude Code lifecycle events. Three of them inject context; one just takes notes. Read left to right.

fires first

01

SESSIONSTART

## A project brief lands before you type a word.

When Claude Code opens in an **engram-initialized project**, the Sentinel reads `.git/HEAD` and queries the graph for god nodes, stats, and top landmines. It composes a ~500-token project brief and hands it to Claude as `additionalContext`.

**Source guard:** fires on `startup` · `clear` · `compact`. Resume is passthrough so context isn't double-injected.

↓ replaces ~4 exploration reads · ≈ -5,000 tok

fires on every prompt

02

USERPROMPTSUBMIT

## Your prompt is pre-queried against the graph.

Keywords are extracted (≥3 chars, non-stopword). If there are ≥2 significant terms AND the graph returns ≥3 matching nodes, engram pre-loads the relevant subgraph as `additionalContext`. Claude starts reasoning with structure already in hand.

**Privacy invariant:** raw prompt text is never written to the hook log. Only the decision (injected / skipped) is recorded. Asserted in tests.

↓ prevents ~1.5 reads per prompt · ≈ -8,000 tok

fires on every tool call

03

PRETOOLUSE

## Read, Edit, Write, and Bash are intercepted.

**Read:** deny + reason with structural summary. File is never actually read — the summary arrives as a system-reminder containing the graph view. **Edit/Write:** allow + landmine warnings (never blocks). **Bash:** strict parser for `cat/head/tail` single-file invocations delegates to the Read handler.

**The money path.** 60% hit rate on mature projects. This is where 18,000 of the 42,500 tokens come from.

↓ replaces full reads · ≈ -18,000 tok

fires on every completion

04

POSTTOOLUSE

## Pure observer — logs, never injects.

After the tool runs, the Sentinel records tool name, file path, output size, success flag, and the PreToolUse decision to `.engram/hook-log.jsonl`. The log is append-only with 10 MB rotation and atomic writes on POSIX.

**Why observer only?** v0.3.0 produces the data; v0.3.1 will use it to self-tune the confidence threshold, mistake boost, and coverage ceiling. Never modifies tool output.

↓ no direct savings · enables self-tuning

02 · THE MONEY PATH, ZOOMED IN

# Every Read goes through *nine checks* before engram decides.

The Read handler is the highest-leverage component. Nine passthrough branches guard against false-positive interceptions. The tenth branch is the one that saves tokens.

PATH 1

### Validate payload shape

tool_name is Read, file_path is a string, offset/limit are not set.

`tool_name` `file_path` `offset>0 · limit>0`

PATH 2

### Content safety

Reject 38 binary extensions (.png .pdf .zip .wasm ...). Reject 10 secret patterns (.env, .env.*, .pem, .key, id_rsa, credentials, secrets.json).

`binary` `secret`

PATH 3

### Resolve context

Normalize path, walk up for `.engram/graph.db`, verify the file lives inside the project root. Exempt paths in `node_modules`, `.git`, `.engram/cache`.

`project root` `outside` `node_modules`

PATH 4

### Kill switch

If `.engram/hook-disabled` exists, passthrough immediately. Used for debugging without uninstalling.

`hook-disabled flag`

PATH 5

### Query graph for file context

Opens `.engram/graph.db`, filters nodes where `sourceFile === relPath`, computes `codeNodeCount` excluding file/module metadata.

`nodes found` `empty graph`

PATH
6

### Staleness check

If `file.mtime > graph.db.mtime`, the graph is stale for this file. Passthrough so Claude sees current contents.

stale

PATH
7

### Confidence threshold

`min(codeNodeCount / 3, 1) × avgNodeConfidence`. Must be ≥ 0.7. Conservative — prefer passthrough over false positive.

≥ 0.7 < 0.7

DENY
→

### Deny + reason with structural summary

Render up to 600 tokens of NODE and EDGE lines sorted by degree, with an escape-hatch footer. Claude Code delivers this as a system-reminder. The file is **never actually read**.

STDOUT — HOOK RESPONSE EXIT 0

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "[engram] Structural
      summary for src/core.ts

      Nodes: 13 | avg extraction
      confidence: 1.00

      NODE init() [function] L48
      NODE query() [function] L131
      NODE path() [function] L145
      NODE godNodes() [function] L159
      NODE stats() [function] L173
      NODE getFileContext() [function] L270
      NODE mistakes() [function] L377
      NODE benchmark() [function] L418
      NODE learn() [function] L182
      NODE getDbPath() [function] L21
      NODE getStore() [function] L25

      Key relationships:
      EDGE core.ts --imports--> query.ts
      EDGE core.ts --imports--> store.ts
      EDGE core.ts --contains--> init()
      EDGE core.ts --contains--> getFileContext()

      Note: engram replaced a full-file
```

```
      read with this structural view to
      save tokens. If you need specific
      lines, Read this file again with
      explicit offset/limit — engram
      passes partial reads through
      unchanged."
  }
}
```

RAW READ: ~4,169 TOK → SUMMARY: ~300 TOK

03 · THE SUBSTRATE

# Six parallel layers feed the *same* session.

engram is one of six persistence layers your Claude Code session leans on. Each one fills a different gap. The Sentinel is the newest; the others have been running for months.

LAYER 01

### engram graph

`.engram/graph.db · SQLite`

Structural code knowledge. AST miner (10 languages, zero LLM cost), git miner (co-change, hot files), session miner (decisions, mistakes). `V0.3 SENTINEL`

LAYER 02

### MemPalace

`~/.mempalace · ChromaDB + SQLite KG`

Semantic memory across 8,646 drawers. Mined from vault, projects, rules, agent memory. Hourly miner + real-time fswatch. `ALWAYS ON`

LAYER 03

### Rule files

`~/.claude/rules/ · 9 files`

brain-os, coding-style, context-rules, failure-playbook, hooks, super-agents, agent-memory, mcp-cli, unified-memory. Loaded every session.

LAYER 04

### Skills

`~/.claude/skills/ · 418 skills`

Pattern-matched, lazy-loaded. Indexed into engram by the skills-miner as `concept` nodes with `triggered_by` edges to keyword concepts.

LAYER 05

### Agents

`~/.claude/agents/ · 101 agents`

Custom + plugin + super commanders. Each spawn reads its own memory file in `~/vault/01-KNOWLEDGE/agent-memory/` before work begins.

LAYER 06

### MCP servers

`mcp-cli wrappers (T1) · direct MCP (T4)`

context7, mempalace, obsidian, render, sentry, playwright, engram, contextplus. CLI wrappers preferred — ~100 tok vs ~5K tok direct call.

04 · REAL MEASUREMENTS

# Not projected. *Measured on real engram code.*

Four representative files from this repo. Three intercepted, one passthrough (correctly — only one exported function, below threshold). 82 % token reduction. 75 % Read hit rate. Same numbers anyone can reproduce with `engram hook-preview`.

| FILE | NODES | RAW READ | SENTINEL | SAVINGS | DECISION |
|---|---|---|---|---|---|
| src/core.ts | 13 | 4,169 | ~300 | -3,869 | DENY |
| src/graph/query.ts | 10 | 4,890 | ~300 | -4,590 | DENY |
| src/intercept/dispatch.ts | 5 | 1,820 | ~300 | -1,520 | DENY |
| src/intercept/handlers/read.ts | 1 | 1,310 | 1,310 | 0 | passthrough |
| **Totals** | **29** | **12,189** | **2,210** | **-9,979** | **-82 %** |

## What the numbers say

SESSION REDUCTION

82 measured

READ HIT RATE

75 measured

PROJECTED (DAY 0)

80 estimated

HOOK COLD START

131 ms

TEST SUITE

1.5 seconds

05 · QUICK REFERENCE

# Seven commands *you'll actually use.*

Everything else is a passthrough branch or a safety invariant. These seven are the surface you interact with.

INSTALL

`engram install-hook`

Adds the four hook entries to `.claude/settings.local.json`. Atomic. Idempotent. Backed up.

DRY-RUN

`engram hook-preview <file>`

Show what the Read handler *would* do for a file. Deny, allow, or passthrough — with the reason.

MEASURE

`engram hook-stats`

Summarize `.engram/hook-log.jsonl`. Per-event, per-tool, per-decision breakdown. Estimated tokens saved.

KILL SWITCH

`engram hook-disable`

Touch `.engram/hook-disabled`. Every handler checks it and passes through. Reversible with `hook-enable`.

REBUILD GRAPH

`engram init`

Re-mine the project. ~40 ms, zero LLM calls, zero tokens used. Run after structural changes.

EXPLICIT QUERY

`engram query "..."`

Direct graph query. BFS with 2,000-token budget. For when the hook didn't catch what you needed.

ROLLBACK

`engram uninstall-hook`

Surgical removal. Preserves every other hook in settings.json. Automatic backup before the write.

UPGRADE

`npm i -g engramx@latest`

Pull the latest from npm. v0.3.1 will tune confidence thresholds from real hook-log data.

# engram *v0.3.0 · Sentinel*

Context as infra for your AI coding tools. Hook-based interception, zero LLM cost, zero cloud, zero native deps.
Published 2026-04-11.

NPM GITHUB RELEASE